

DBMS PLACEMENT PREPARATION NOTES

Modules 14–17 • Serializability • Two-Phase Locking • Timestamp Ordering • Recovery

Continuation: These four topics come immediately after Transactions & ACID. The focus is concurrency control and database recovery.

MODULE 14 — SERIALIZABILITY

Schedule: The order in which operations of multiple transactions are executed.

Serial schedule: T1 finishes, then T2 starts. **Concurrent schedule:** operations of transactions are interleaved.



Concurrent execution interleaves operations.

1. Why Serializability?

Concurrent execution can improve performance, but it may produce an incorrect database state. A schedule is considered **serializable** when its result is equivalent to a correct serial execution.

2. Conflicting Operations

Two operations conflict when they belong to different transactions, access the **same data item**, and at least one operation is a write.

Operations	Conflict?	Reason
R(A), R(A)	No	Both only read.
R(A), W(A)	Yes	Same item; one writes.
W(A), R(A)	Yes	Same item; one writes.
W(A), W(A)	Yes	Both write the same item.

3. Conflict Serializability

A schedule is conflict-serializable if it can be rearranged into a serial schedule by swapping only **non-conflicting** operations.

4. Precedence / Serialization Graph

Create one node for each transaction. Draw $T_i \rightarrow T_j$ when a conflicting operation of T_i occurs before the conflicting operation of T_j .



No cycle → conflict-serializable. A cycle → not conflict-serializable.

5. View Serializability

A schedule is view-serializable when it has the same important read-from relationships and final-write behavior as some serial schedule. It is more general than conflict serializability.

Conflict Serializability	View Serializability
Uses conflicting operations.	Uses read-from and final-write relationships.
Precedence graph is a common test.	More general and harder to test.
Every conflict-serializable schedule is view-serializable.	Not every view-serializable schedule is conflict-serializable.

■ Placement memory: **Precedence graph + NO cycle = conflict serializable**. Read-read operations never create a conflict.

MODULE 15 — TWO-PHASE LOCKING (2PL)

Lock: Permission/control mechanism used before accessing a data item during concurrent execution. 2PL divides locking into two phases.



Acquire locks → stop acquiring → release locks → commit/abort

1. Growing Phase

A transaction can acquire locks but cannot release any lock.

2. Shrinking Phase

A transaction can release locks but cannot acquire any new lock.

3. Lock Point

The moment when the transaction has acquired its final lock. After this point it enters the shrinking phase.

4. Basic 2PL

Follows growing and shrinking phases. It guarantees **conflict serializability**, but it can still cause deadlocks and cascading rollbacks.

5. Strict 2PL

The transaction keeps its **exclusive/write locks** until commit or abort. This prevents other transactions from reading uncommitted written data and helps avoid cascading rollback.

6. Rigorous 2PL

The transaction keeps **all locks**—shared and exclusive—until commit or abort.

7. Conservative / Static 2PL

The transaction obtains all required locks before it starts. Because it does not wait for locks during execution, it can prevent deadlocks; however, all required locks must be known beforehand.

Variant	Remember
Basic 2PL	Growing + shrinking; conflict-serializable.
Strict 2PL	Hold X/write locks until commit/abort.
Rigorous 2PL	Hold S/read and X/write locks until commit/abort.
Conservative 2PL	Acquire all needed locks before execution; prevents deadlock.

8. Shared vs Exclusive Lock

Lock	Meaning
S (Shared)	Used for reading; compatible shared locks may coexist.
X (Exclusive)	Used for writing; conflicts with S and X locks on the same item.

■ Trap: 2PL means **two phases**, not two locks. Basic 2PL guarantees serializability but does not automatically prevent deadlock.

MODULE 16 — TIMESTAMP ORDERING PROTOCOL

Timestamp: A unique value assigned to a transaction to represent its order. Older transaction = smaller timestamp.

Timestamp Ordering controls concurrent operations by requiring them to respect timestamp order instead of using locks.

1. RTS(X) and WTS(X)

RTS(X): largest timestamp of a transaction that successfully read X. **WTS(X):** largest timestamp of a transaction that successfully wrote X.

Term	Meaning
TS(T)	Timestamp of transaction T.
RTS(X)	Latest successful read timestamp for X.
WTS(X)	Latest successful write timestamp for X.

2. Basic Read Rule

When T wants to read X, a common timestamp-ordering check is: if **TS(T) < WTS(X)**, the read would violate timestamp order, so T is rolled back. Otherwise the read is allowed and RTS(X) may be updated.

3. Basic Write Rule

When T wants to write X, if **TS(T) < RTS(X)** or **TS(T) < WTS(X)**, the write violates timestamp order and T is rolled back under the basic protocol. Otherwise the write is allowed and WTS(X) is updated.

4. Thomas' Write Rule

Thomas' Write Rule is a modification of timestamp ordering. In a case where an older transaction attempts a write that has already been made obsolete by a newer write, the obsolete write can be ignored rather than necessarily rolling back the transaction.

5. Example

Suppose TS(T1)=5, TS(T2)=10. T1 is older. If WTS(A)=10 and T1 later tries to read A, TS(T1) < WTS(A), so the basic read rule rejects the read and T1 is rolled back.

6. Timestamp Ordering vs 2PL

2PL	Timestamp Ordering
Uses locks.	Uses timestamps.
Waiting for locks may occur.	Violating operation may cause rollback.
Deadlock is possible.	Basic protocol is not lock-based, so lock deadlock is avoided.
Growing/shrinking phases.	Read/write timestamp checks.

■ Placement memory: **RTS** = read timestamp; **WTS** = write timestamp; smaller **TS** = older transaction.

MODULE 17 — RECOVERY & RECOVERABILITY

Recovery: Restoring the database to a correct state after failures.



Recovery must preserve committed work and remove invalid uncommitted work.

1. Common Failure Types

Failure	Example
Transaction failure	Constraint violation, invalid operation, explicit abort.
System crash	Power failure or DBMS/OS crash.
Media failure	Disk/storage damage or corruption.

2. Recoverable Schedule

If T2 reads data written by T1, T2 must not commit before T1 commits. This ensures that T2 does not permanently commit based on a transaction that may later abort.

3. Cascading Rollback

If T2 reads uncommitted data written by T1 and T1 aborts, T2 may also need to abort. This chain of rollbacks is a **cascading rollback**.

4. Cascadeless Schedule

A cascadeless schedule allows transactions to read only **committed** data. Therefore, an abort does not create a rollback chain caused by dirty reads.

5. Strict Schedule

A strict schedule does not allow another transaction to read or write a data item written by T until T commits or aborts. It provides a strong form of recovery safety.

Property	Easy way to remember
Recoverable	Dependent transaction commits after the transaction it read from commits.
Cascadeless	Read only committed data.
Strict	Do not read/write another transaction's written item until it commits/aborts.

6. Log-Based Recovery

Log: Record of transaction actions used during recovery. **UNDO:** reverse changes that should not remain. **REDO:** reapply changes that should remain but may not have reached stable storage.

7. Write-Ahead Logging (WAL)

Before the modified database page is written to stable storage, the corresponding log information is written first. This allows the recovery system to undo/redo operations after a failure.

8. Checkpoint

A checkpoint is a recovery marker that reduces the amount of log the DBMS needs to examine after a crash.

9. Deferred vs Immediate Update

Deferred Update	Immediate Update
Database changes are delayed until commit.	Database pages may be updated before commit.
Recovery mainly needs REDO for committed work.	Recovery may require UNDO for uncommitted work and REDO for committed work.

■ Placement questions

- Recoverable? → Dependent transaction commits after the transaction it read from commits.
- Cascading rollback? → One failure forces dependent transactions to roll back.
- Cascadeless? → No transaction reads uncommitted data.
- WAL? → Log first, database page later.
- Checkpoint? → Reduces recovery work.

FINAL 1-MINUTE REVISION — MODULES 14–17

Topic	Remember
Serializability	Concurrent schedule equivalent to a correct serial execution.
Precedence graph	No cycle → conflict-serializable.
2PL	Growing phase + shrinking phase.
Strict 2PL	Hold X locks until commit/abort.
Rigorous 2PL	Hold all locks until commit/abort.
Conservative 2PL	Acquire all required locks before execution.
Timestamp Ordering	Use timestamps to control operation order.
RTS / WTS	Latest successful read / write timestamps.
Recoverable	Dependent transaction commits after source transaction commits.
Cascadeless	Reads only committed data.
Strict schedule	Protects written items until commit/abort.
WAL	Write log before corresponding database page.
Checkpoint	Reduces recovery work.

Practice priority: Solve precedence-graph questions, compare 2PL variants, apply timestamp read/write rules, and identify recoverable/cascadeless/strict schedules. These are much easier after understanding the terminology first.